DBMS – ADVANCED SQL QUERY SYSTEM

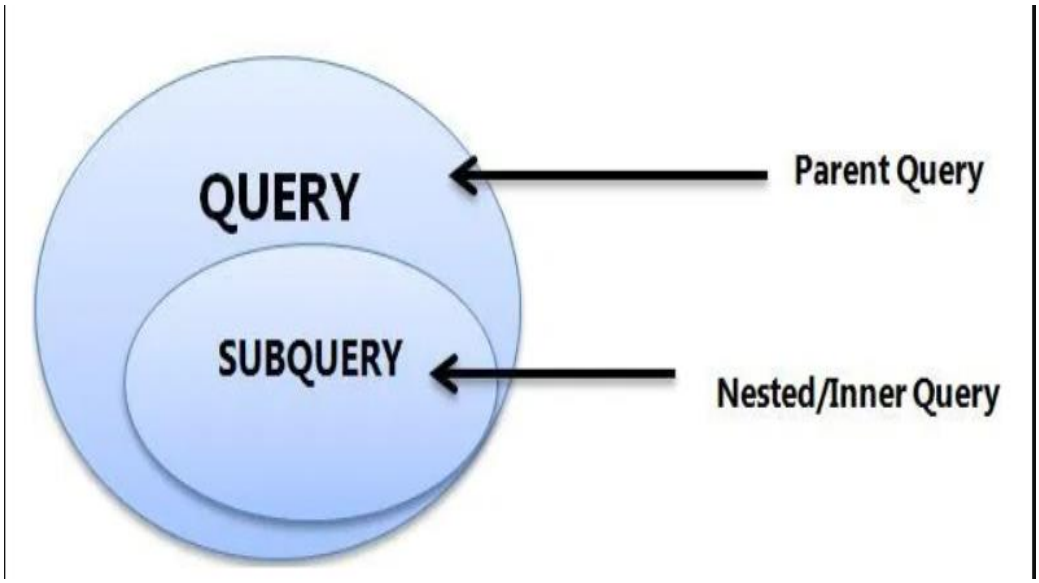
Project Title: Myntra E-Commerce Order Management System

Objectives

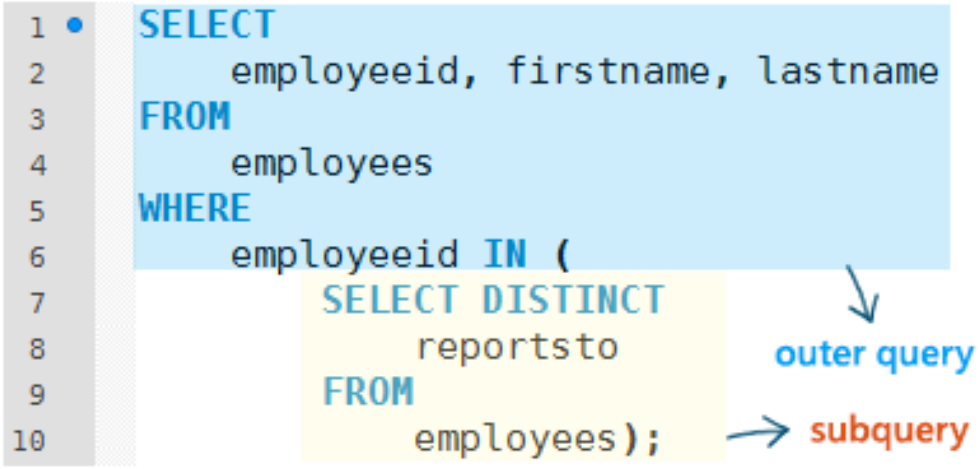
1. Implement subqueries and nested queries.
2. Find products above the average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare advanced SQL reports.

1. Subqueries and Nested Queries

A subquery is a query written inside another SQL query. The inner query is executed first, and its result is used the outer query.



Basic Structure



Type	Description
Single-row subquery	Returns one value
Multiple-row subquery	Returns multiple values
Nested subquery	A subquery inside another subquery
Correlated subquery	Inner query depends on the outer query

Example

Find orders whose amount is greater than the average order amount.

```
SQL> SELECT Order_ID, Customer_ID, Total_Amount
  2  FROM Orders
  3  WHERE Total_Amount >
  4        (SELECT AVG(Total_Amount)
  5         FROM Orders);

  ORDER_ID CUSTOMER_ID TOTAL_AMOUNT
---------- ----------- ------------
       502         102       3398.3
       504         104      5699.05
```

Explanation

The inner query:

SELECT AVG(Total_Amount) FROM Orders;

calculates the average order amount.

The outer query then displays orders whose Total_Amount is greater than that average.

2. Find Products Above Average Price

A subquery can be used to compare each product's price with the average product price.

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price >
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE
----------
       203
Running Shoes
      3499

       204
Analog Watch
      5999

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------
     PRICE
----------
```

Explanation

The inner query ***SELECT AVG(Price) FROM Product;***

calculates the average price (2939). The outer query displays products whose price is greater than the calculated average.

2.1 Products Below Average Price

```
SQL> SELECT Product_ID, Product_Name, Price
  2  FROM Product
  3  WHERE Price <
  4        (SELECT AVG(Price)
  5         FROM Product);

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
     PRICE
----------
       201
Cotton Shirt
      1399

       202
Floral Dress
      1999

PRODUCT_ID
----------
PRODUCT_NAME
----------------------------------------
     PRICE
----------

       205
Backpack
      1799
```

3. Identify Customers with Maximum Purchases

We can use SUM() along with a subquery to identify the customer who has the highest total purchase amount.

Step 1: Calculate Customer-wise Purchase

```
SQL> SELECT Customer_ID,
  2         SUM(Total_Amount) AS Total_Purchase
  3  FROM Orders
  4  GROUP BY Customer_ID;

CUSTOMER_ID TOTAL_PURCHASE
----------- --------------
        101        1231.12
        102         3398.3
        103         2799.2
        104        5699.05
        105         1619.1
        106        2462.24

6 rows selected.
```

Step 2: Find the Maximum Purchase

```
SQL> SELECT MAX(Total_Purchase)
  2  FROM (
  3      SELECT Customer_ID,
  4             SUM(Total_Amount) AS Total_Purchase
  5      FROM Orders
  6      GROUP BY Customer_ID
  7  );

MAX(TOTAL_PURCHASE)
-------------------
            5699.05
```

The inner query calculates the total purchase for each customer. The outer query finds the maximum value among those totals.

3.1 Display Customer Details with Maximum Purchase

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         SUM(o.Total_Amount) AS Total_Purchase
  4  FROM Customer c
  5  JOIN Orders o
  6      ON c.Customer_ID = o.Customer_ID
  7  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
  8  HAVING SUM(o.Total_Amount) = (
  9      SELECT MAX(Total_Purchase)
 10      FROM (
 11          SELECT SUM(Total_Amount) AS Total_Purchase
 12          FROM Orders
 13          GROUP BY Customer_ID
 14      ));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        104
Anitha Suresh
       5699.05
```

Explanation

- The innermost query SELECT SUM(Total_Amount) FROM Orders GROUP BY Customer_ID; calculates the purchase amount for every customer.
- The next query SELECT MAX(Total_Purchase) finds the highest purchase amount.
- The main query displays the customer whose purchase amount matches that maximum.

4. Complex Business Queries

Subqueries can be combined with joins and aggregate functions to solve business-related problems.

4.1 Customers Who Placed Orders Above Average Order Value

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         o.Order_ID,
  4         o.Total_Amount
  5  FROM Customer c
  6  JOIN Orders o
  7      ON c.Customer_ID = o.Customer_ID
  8  WHERE o.Total_Amount >
  9  (
 10      SELECT AVG(Total_Amount)
 11      FROM Orders
 12  )
 13  ORDER BY o.Total_Amount DESC, o.Order_ID;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID TOTAL_AMOUNT
---------- ------------
       104
Anitha Suresh
       504      5699.05
```

Purpose

This report identifies customers who placed orders whose value is greater than the overall average order value**.**

5. Find Products Whose Sales Are Above Average

The total sales of each product can be calculated from Order_Details.

```
SQL> SELECT p.Product_ID,
  2         p.Product_Name,
  3         SUM(oi.Total_Price) AS Total_Sales
  4  FROM Product p
  5  JOIN Order_Items oi
  6      ON p.Product_ID = oi.Product_ID
  7  GROUP BY p.Product_ID, p.Product_Name
  8  HAVING SUM(oi.Total_Price) > (
  9      SELECT AVG(Product_Sales)
 10      FROM (
 11          SELECT SUM(Total_Price) AS Product_Sales
 12          FROM Order_Items
 13          GROUP BY Product_ID
 14      ))
 15  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME                                                                                          TOTAL_SALES
---------- ---------------------------------------------------------------------------------------------------- -----------
       204 Analog Watch                                                                                            5699.05
       201 Cotton Shirt                                                                                            3693.36
```

6. Customer Purchase Report

A detailed customer purchase report can combine multiple tables.

```
SQL> SELECT c.Customer_ID,
  2         c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3         COUNT(o.Order_ID)   AS Total_Orders,
  4         SUM(o.Total_Amount) AS Total_Purchase,
  5         AVG(o.Total_Amount) AS Average_Order_Value
  6  FROM Customer c
  7  JOIN Orders o
  8      ON c.Customer_ID = o.Customer_ID
  9  GROUP BY c.Customer_ID, c.First_Name, c.Last_Name
 10  ORDER BY Total_Purchase DESC;

CUSTOMER_ID CUSTOMER_NAME                                                                                          TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_ORDER_VALU
E
----------- ------------------------------------------------------------------------------------------------------ ------------ -------------- ------------------
-
        104 Anitha Suresh                                                                                                 1       5699.05            5699.0
5
```

Report Contains

- Customer ID
- Customer Name
- Number of orders
- Total purchase amount
- Average order value

7. Advanced Product Sales Report

This report combines product and order details.

```
SQL> SELECT p.Product_ID,
  2         p.Product_Name,
  3         COUNT(oi.Order_ID)   AS Number_of_Orders,
  4         SUM(oi.Quantity)     AS Total_Quantity_Sold,
  5         SUM(oi.Total_Price)  AS Total_Sales,
  6         AVG(oi.Unit_Price)   AS Average_Price
  7  FROM Product p
  8  JOIN Order_Items oi
  9      ON p.Product_ID = oi.Product_ID
 10  GROUP BY p.Product_ID, p.Product_Name
 11  ORDER BY Total_Sales DESC;

PRODUCT_ID PRODUCT_NAME         NUMBER_OF_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
---------- -------------------- ---------------- ------------------- ----------- -------------
       204 Analog Watch                        1                   1     5699.05          5999
       201 Cotton Shirt                        2                   3     3693.36          1399
       202 Floral Dress                        1                   2      3398.3          1999
       203 Running Shoes                       1                   1      2799.2          3499
       205 Backpack                            1                   1      1619.1          1799
```

8. Category-wise Advanced Sales Report

```
SQL> SELECT cat.Category_ID,
  2         cat.Category_Name,
  3         COUNT(DISTINCT oi.Order_ID) AS Total_Orders,
  4         SUM(oi.Quantity)            AS Total_Quantity_Sold,
  5         SUM(oi.Total_Price)         AS Total_Sales,
  6         AVG(oi.Unit_Price)          AS Average_Price
  7  FROM Category cat
  8  JOIN Product p
  9      ON cat.Category_ID = p.Category_ID
 10  JOIN Order_Items oi
 11      ON p.Product_ID = oi.Product_ID
 12  GROUP BY cat.Category_ID, cat.Category_Name
 13  ORDER BY Total_Sales DESC;

CATEGORY_ID CATEGORY_NAME        TOTAL_ORDERS TOTAL_QUANTITY_SOLD TOTAL_SALES AVERAGE_PRICE
----------- -------------------- ------------ ------------------ ----------- -------------
        104 Watches                         1                  1     5699.05          5999
        101 Men Clothing                    2                  3     3693.36          1399
        102 Women Clothing                  1                  2      3398.3          1999
        103 Footwear                        1                  1      2799.2          3499
        105 Bags                            1                  1      1619.1          1799
```